

LAB 7

Demonstrating CSS Selectors

Subject: Web Designing

Lab Focus: CSS Selectors — Basic, Grouping, Combinator & Attribute Selectors

Tools Required: Any text editor (VS Code / Notepad) and a Web Browser

Prerequisite Knowledge: Basic HTML structure and inline/internal CSS syntax

Topics Covered in This Lab

1. Part A — Basic Selectors: Element, Class, and ID Selector
2. Part B — Group Selector, Universal Selector, and Descendant Selector
3. Part C — Child Selector, Adjacent Sibling Selector, and Attribute Selector

Lab 7: Demonstrate the Selectors

Objective: To understand what CSS selectors are and how different types of selectors — basic, group, universal, descendant, child, adjacent sibling, and attribute — are used to target HTML elements precisely and apply styles to them.

What is a CSS Selector?

A **CSS selector** is a pattern used to "select" the HTML element(s) that a style rule should apply to. Every CSS rule follows the structure: `selector { property: value; }`. Selectors are what make CSS powerful — they let a designer target a single element, a group of elements, elements of a specific type, or elements based on their position/relationship in the HTML document (the "DOM tree").

Part (A): Basic Selectors — Element, Class, and ID Selector

Explanation

Selector	Syntax	Description
Element Selector	<code>p { }</code>	Selects <u>every</u> instance of a given HTML tag on the page. Applies the same style to all matching elements — e.g. all <code><p></code> paragraphs.
Class Selector	<code>.className { }</code>	Selects all elements carrying a specific <code>class="className"</code> attribute. A class can be reused on many different elements, and one element can carry multiple classes.
ID Selector	<code>#idName { }</code>	Selects the single element carrying <code>id="idName"</code> . IDs must be unique within a page — only one element should use a given id.

Precedence tip: ID selectors are more specific than class selectors, which are more specific than element selectors. So when rules conflict, ID styles override class styles, and class styles override plain element styles.

Code Example

```
<!DOCTYPE html>
<html>
<head>
<style>
  /* Element selector - targets ALL <p> tags */
  p {
    color: navy;
    font-family: Arial, sans-serif;
  }

  /* Class selector - targets elements with class="highlight" */
  .highlight {
    background-color: yellow;
    font-weight: bold;
  }

  /* ID selector - targets the single element with id="main-heading" */
  #main-heading {
    color: darkred;
    text-align: center;
  }
</style>
</head>
<body>
  <h1 id="main-heading">Basic CSS Selectors</h1>
  <p>This paragraph uses the element selector styling.</p>
  <p class="highlight">This paragraph is also highlighted using a class selector.</p>
</body>
</html>
```

Output/Effect: The heading "Basic CSS Selectors" appears centered in dark red (from the ID selector). Both paragraphs appear in navy blue Arial text (from the element selector), and the second paragraph additionally has a yellow background with bold text (from the class selector) layered on top.

Part (B): Group Selector, Universal Selector, and Descendant Selector

Explanation

Selector	Syntax	Description
Group Selector	<code>h1, p { }</code>	Applies the <u>same</u> style block to multiple different selectors at once, separated by commas — avoids repeating identical CSS rules.
Universal Selector	<code>* { }</code>	Selects <u>every single element</u> on the page. Commonly used for global resets, e.g. removing default margin/padding from all elements.
Descendant Selector	<code>div p { }</code>	Selects elements that are nested <u>anywhere inside</u> another specified element (at any depth), written as two or more selectors separated by a space. Here, it selects all <code><p></code> tags that appear inside a <code><div></code> , no matter how deeply nested.

Code Example

```
<!DOCTYPE html>
<html>
<head>
<style>
  /* Universal selector - resets margin/padding on every element */
  * {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
  }

  /* Group selector - same style applied to h1 AND p */
  h1, p {
    font-family: "Segoe UI", sans-serif;
    color: #333;
  }

  /* Descendant selector - only <p> tags that are inside a <div> */
  div p {
    color: green;
    font-style: italic;
  }
</style>
</head>
<body>
  <h1>Group, Universal & Descendant Selectors</h1>
  <p>This paragraph is directly in the body (not in a div).</p>
  <div>
    <p>This paragraph is inside a div, so the descendant selector turns it green.</p>
  </div>
</body>
</html>
```

Output/Effect: The heading and both paragraphs share the same font family and base grey color (group selector). However, only the paragraph nested inside the `<div>` turns green and italic, because it alone matches the `div p` descendant rule; the paragraph directly in `<body>` stays the plain grey color.

Part (C): Child, Adjacent Sibling, and Attribute Selectors

Explanation

Selector	Syntax	Description
Child Selector	<code>ul > li { }</code>	Selects only the <u>direct</u> (immediate) children of an element — unlike the descendant selector, it will not match elements nested deeper. Here, only <code></code> elements that are direct children of a <code></code> are selected, not <code></code> inside a nested <code></code> .
Adjacent Sibling Selector	<code>h2 + p { }</code>	Selects an element that comes <u>immediately after</u> another specified element, at the same level (both must share the same parent). Here, it selects only the very first <code><p></code> that appears directly after an <code><h2></code> .
Attribute Selector	<code>input[type="text"] { }</code>	Selects elements based on the presence or value of an HTML attribute. Here, it targets only <code><input></code> elements whose <code>type</code> attribute equals <code>"text"</code> , leaving other input types (checkbox, submit, etc.) unaffected.

Child (>) vs Descendant (space): `div p` matches a `<p>` anywhere inside the `<div>`, even several levels deep. `div > p` matches a `<p>` only if it is a direct child of that exact `<div>`.

Code Example

```
<!DOCTYPE html>
<html>
<head>
<style>
  /* Child selector - only DIRECT <li> children of <ul> */
```

```

ul > li {
  color: blue;
  list-style-type: square;
}

/* Adjacent sibling selector - the <p> immediately after an <h2> */
h2 + p {
  font-weight: bold;
  color: crimson;
}

/* Attribute selector - only <input> elements of type="text" */
input[type="text"] {
  border: 2px solid teal;
  padding: 6px;
}
</style>
</head>
<body>

  <h2>Child Selector Demo</h2>
  <ul>
    <li>Direct child item 1</li>
    <li>Direct child item 2
      <ul> <ol> here ol used not ul
        <li>Nested item (NOT styled by ul > li)</li>
      </ul> </ol> here ol used not ul
    </li>
  </ul>

  <h2>Adjacent Sibling Demo</h2>
  <p>This paragraph is styled because it directly follows the h2 above.</p>
  <p>This second paragraph is NOT styled (not immediately after an h2).</p>

  <h2>Attribute Selector Demo</h2>
  <form>
    <input type="text" placeholder="Styled text input"><br><br>
    <input type="checkbox"> Not styled (checkbox)
  </form>

</body>
</html>

```

Output/Effect: The two top-level list items turn blue with square bullets, but the nested `<li>` inside the inner `<ul>` stays in default black text since it is not a direct child of the outer `<ul>`. Only the paragraph immediately following the second `<h2>` turns bold crimson. The text input box gets a teal border and padding, while the checkbox input is left completely unstyled.

Quick Revision — Lab 7 Summary

Selector	Syntax	Matches
Element	<code>p</code>	All <code><p></code> elements
Class	<code>.className</code>	All elements with that class
ID	<code>#idName</code>	The one element with that id
Group	<code>h1, p</code>	All <code><h1></code> AND all <code><p></code> elements
Universal	<code>*</code>	Every element on the page
Descendant	<code>div p</code>	Any <code><p></code> nested anywhere inside a <code><div></code>
Child	<code>ul > li</code>	Only direct <code></code> children of <code></code>
Adjacent Sibling	<code>h2 + p</code>	The <code><p></code> immediately following an <code><h2></code>
Attribute	<code>input[type="text"]</code>	Only <code><input></code> with that exact attribute value

Viva / Practice Questions

1. What is the difference between a class selector and an ID selector?
2. Why can an ID be used only once per page while a class can be reused?
3. What does the universal selector (`*`) do, and why is it often used with margin/padding resets?
4. Explain the difference between the descendant selector (space) and the child selector (`>`) with an example.
5. What does the adjacent sibling selector (`+`) select? Will `h2 + p` match a paragraph that is two elements after the `h2`?
6. Write an attribute selector that targets all `<input>` elements with `type="password"`.

7. Rank element, class, and ID selectors in order of CSS specificity (lowest to highest).